Functional Programming Practical Week 7 Part 2

Option Either Result and Functional Error Handling

30 additional practical questions and tasks based on the detailed course plan. Represent missing values and expected failures explicitly. Use Map and Bind to transform or connect successful computations and refactor exception-heavy code.

Use C# 12 and .NET 8. Define Option<T> with Some(T) and None cases, Result<T> with Ok(T) and Error(string), and Either<L, R> with Left(L) and Right(R). You may use record hierarchies and pattern matching. These types are implemented by you, not assumed to be built into C#. Unless stated otherwise, expected input failures should return values.

Optional and ungraded practice. During defence, you will need to implement one of these tasks or a similar task. Any practical task may appear in the endterm.

Basic tasks – Questions 01–05

Question 01 An explicit optional value

Define Option<T> and its Some and None cases. Write a function that returns Some for a nonblank string and None for null, empty or whitespace-only input. Trim accepted values.

Checks and discussion: " Ali " becomes Some("Ali"). null, "" and " " become None.

Question 02 A missing lookup result

Implement FindName(dictionary, id) returning Option<string>. Use None for an absent key rather than an empty string or an exception.

Checks and discussion: An existing key whose stored value is "" returns Some(""). An absent key returns None.

Question 03 Handling both cases

Implement Describe(Option<int>) returning "value:<number>" for Some and "missing" for None. Use pattern matching and do not cast blindly to Some.

Checks and discussion: Some(0) returns "value:0". None returns "missing".

Question 04 A default value

Implement GetOrElse<T>(Option<T>, T fallback). State the difference between None and Some containing the default value of T.

Checks and discussion: Some(0) with fallback 99 returns 0. None with fallback 99 returns 99.

Question 05 Map for an optional value

Implement Map<T, U>(Option<T>, Func<T, U>). Apply the function only to Some and preserve None.

Checks and discussion: Some(3) mapped with x => x*2 becomes Some(6). The callback is not called for None.

Functional Programming Practical Week 7 Part 2

Basic tasks – Questions 06–10

Question 06 Parsing as Option

Implement ParseInt(string?) returning Option<int> using TryParse with invariant culture. Explain what information this API loses compared with a result carrying an error.

Checks and discussion: "0" becomes Some(0). "abc" and null become None.

Question 07 Map does not flatten a result

Apply Map to Option<string> using ParseInt from Question 6. Identify the resulting nested type and explain why it is inconvenient for a parsing pipeline.

Checks and discussion: Some("12") produces Some(Some(12)). Some("abc") produces Some(None). Outer None remains None.

Question 08 Bind for an optional value

Implement Bind<T, U>(Option<T>, Func<T, Option<U>>). Use it to parse an optional string without nesting Option values.

Checks and discussion: Some("12") becomes Some(12). Some("abc") and outer None both become None.

Question 09 A safe first element

Implement FirstOption<T>(IEnumerable<T>) without materialising the sequence. Enumerate only enough to distinguish an empty input from a first element.

Checks and discussion: Empty input returns None. A sequence starting with 0 returns Some(0). Dispose the enumerator.

Question 10 Filtering an optional value

Implement Where<T>(Option<T>, Func<T, bool>) that keeps a Some value only when its predicate succeeds.

Checks and discussion: Some(4) with an even predicate stays Some(4). Some(3) becomes None. None does not invoke the predicate.

Functional Programming Practical Week 7 Part 2

Applying the techniques – Questions 11–15

Question 11 A result with an error reason

Define Result<T>. Implement ParsePositiveInt(string?) returning a distinct error for an invalid integer format and for a parsed value <= 0.

Checks and discussion: "5" returns Ok(5). "five" returns a format error. "0" returns a range error.

Question 12 Transforming a successful result

Implement Map<T, U> for Result<T>. Preserve the original error message and transform only Ok.

Checks and discussion: Ok(3) mapped with x => x*x becomes Ok(9). Error("format") stays an error with the same message.

Question 13 Connecting fallible operations

Implement Bind<T, U> for Result<T>. Use it to connect ParsePositiveInt with a function that rejects numbers above 100.

Checks and discussion: "20" succeeds, "101" produces an upper-bound error, and "abc" stops at parsing.

Question 14 Refactoring Parse and catch

Replace a function that calls int.Parse inside try/catch and returns 0 on any failure. Return Result<int> instead and make valid zero distinguishable from invalid text.

Checks and discussion: "0" returns Ok(0). "abc" returns Error. Explain why returning zero hid the failure.

Question 15 Refactoring a missing dictionary entry

Replace direct dictionary indexing used for a normal user lookup with TryGetValue and Result<string>. Include the requested id in the error.

Checks and discussion: A missing user returns an explanatory Error. Existing users keep their stored names unchanged.

Functional Programming Practical Week 7 Part 2

Applying the techniques – Questions 16–20

Question 16 Keeping the first failure

Compose three Result<int> functions with Bind. Specify that the first error is returned and later functions are not invoked.

Checks and discussion: Use diagnostic callbacks to show that failure at stage two prevents stage three. Remove diagnostic mutation from the final business functions.

Question 17 A result versus an exception

Classify malformed user input, a missing optional value, division by zero in user data and an unexpected broken internal invariant. Propose a contract for each and justify it.

Checks and discussion: Provide one short call-site example showing how a returned Error differs from an exception that propagates.

Question 18 Converting Option into Result

Implement ToResult<T>(Option<T>, string missingMessage). Convert Some to Ok and None to Error.

Checks and discussion: Some(0) becomes Ok(0). None uses exactly the supplied message. Do not reinterpret zero as missing.

Question 19 A lazy fallback

Implement GetOrElse<T>(Option<T>, Func<T> fallback). Invoke fallback only for None, unlike an already evaluated fallback value.

Checks and discussion: Use a call counter to prove Some does not compute the fallback and None computes it once per GetOrElse call.

Question 20 Validation of two fields

Validate a nonblank username and a quantity > 0 independently using Result values. Implement an API that collects both errors instead of stopping at the first one.

Checks and discussion: Blank username and quantity 0 produce two errors. State the return model, including the all-valid case.

Functional Programming Practical Week 7 Part 2

Integration and review – Questions 21–25

Question 21 Either with a typed error

Define Either<ValidationError, int>, where ValidationError contains Code and Message. Implement integer parsing using Left for failure and Right for success.

Checks and discussion: Invalid text returns code "format". Valid "12" returns Right(12). Define ValidationError yourself.

Question 22 Map on the Right case

Implement a Right-oriented Map for Either<L, R>. Keep Left unchanged and transform Right.

Checks and discussion: Right(2) mapped with x => x+1 becomes Right(3). The same Left error is preserved.

Question 23 Bind with one error type

Implement Right-oriented Bind for Either<L, R>. Connect two operations that share L as their error type.

Checks and discussion: A Left skips the callback. A Right can become a new Left or a new Right, with no nested Either.

Question 24 A safe division pipeline

Parse numerator and denominator into Result<decimal>, then return their quotient. Treat malformed fields and a zero denominator as expected failures.

Checks and discussion: "10" and "2" give Ok(5m). "10" and "0" give a zero-denominator error. Use invariant numeric parsing.

Question 25 A sequence of results

Implement Sequence<T>(IEnumerable<Result<T>>) returning Result<T[]> with the first error or all successful values in order. Enumerate the source once.

Checks and discussion: Ok(1), Error("bad"), Ok(3) returns Error("bad"). Empty input returns Ok with an empty array.

Functional Programming Practical Week 7 Part 2

Integration and review – Questions 26–30

Question 26 Collecting row errors

Implement a batch parser that reports every invalid row with its one-based row number. Return accepted values and errors separately in an explicit model.

Checks and discussion: For valid, invalid, valid rows, preserve both valid values and report row 2. State that this is a partial-import policy.

Question 27 An exception boundary

Wrap reading a text file in Result<string[]> and translate FileNotFoundException and UnauthorizedAccessException into different error codes. Allow unexpected exceptions to propagate.

Checks and discussion: Explain how an empty existing file differs from a missing file. Do not catch Exception and report success.

Question 28 Recovery from a specific failure

Implement recovery that substitutes a default configuration only for a "not_found" error. Preserve parse and permission errors.

Checks and discussion: A missing configuration uses the default. A malformed configuration remains an error instead of silently using the default.

Question 29 Behaviour of Map and Bind

For your Result implementation, demonstrate Map with identity, Bind with a function returning Ok, and Bind beginning with Error. Explain the expected behaviour using concrete values.

Checks and discussion: Use Ok(5) and Error("bad"). State that callbacks should be pure when comparing equivalent compositions.

Question 30 Refactoring an exception-heavy module

Choose a small registration or import workflow with at least three expected failure cases. Refactor it to Result or Either and keep user-facing formatting at the boundary.

Checks and discussion: Show success, every expected failure and an unexpected exception. Explain why your chosen error type fits the caller.